

EduBridge

Analyse Architecturale Complete pour Soutenance PFE

Plateforme candidats <-> instituts prives tunisiens

Frontend React / Backend Node.js / Microservice FastAPI OCR

- Architecture complete : frontend / backend / microservice
- Flux detailles : auth JWT, candidature, upload, OCR
- Analyse fichier par fichier des composants critiques
- 25+ questions jury avec reponses pour la soutenance
- Forces, faiblesses et vocabulaire technique

Document de preparation - Mode READ-ONLY - Mai 2026

EduBridge -- Analyse Architecturale Complete pour Soutenance PFE

Document genere pour preparer la soutenance de Projet de Fin d'Etudes.

Mode READ-ONLY -- aucun fichier du projet n'a ete modifie.

TABLE DES MATIERES

1. Vue globale du systeme
2. Architecture generale -- Diagrammes
3. Flux detailles end-to-end
4. Explication dossier par dossier
5. Explication fichier par fichier
6. Analyse du code ligne par ligne
7. Questions probables du jury -- avec reponses
8. Forces et faiblesses du projet
9. Vocabulaire soutenance

1. VUE GLOBALE DU SYSTEME

1.1 Qu'est-ce qu'EduBridge ?

EduBridge est une plateforme SaaS de mise en relation entre :

- Candidats (etudiants cherchant une formation)
- Instituts prives tunisiens (ecoles qui publient leurs programmes)
- Administrateurs (gestion, validation, supervision)

Le projet est compose de trois composants independants :

Composant	Technologie	Role
backend/	Node.js / Express / PostgreSQL	API REST -- coeur metier
frontend/	React / Vite / TypeScript	Interface utilisateur SPA
diploma-verifier/	Python / FastAPI	Microservice OCR de verification document

1.2 Les trois roles metier

EDUBRIDGE		
CANDIDAT	INSTITUT	ADMIN
S'inscrit	Publie des	Gere tout
Cherche	programmes	Valide les
Candidate	Recoit les	instituts
Suit son	candidatures	Supervise
dossier	Les traite	Statistiques

```

+-----+
| NAVIGATEUR WEB |
+-----+
|
| FRONTEND -- React SPA (Vite) |
|
| +-----+ +-----+ +-----+ +-----+ |
| |AuthContext| | Hooks | | Pages | |Routes | |
| | (JWT) | |(fetch) | |(UI) | |(react- | |
| +-----+ +-----+ +-----+ |router) | |
|
| | | | | +-----+ |
| +-----+ |
|
| +-----v-----+ |
| | api.ts (axios) | Client HTTP central |
| | + intercepteurs | JWT auto-inject |
| +-----+-----+ |
+-----+
|
| HTTPS / JSON
|
+-----v-----+
| BACKEND -- Node.js / Express |
|
| +-----+ +-----+ +-----+ +-----+ |
| |Middleware| | Routes | |Controller| | Services | |
| | (JWT, | | (REST) | | (HTTP) | | (metier) | |
| | upload) | | | | | | |
| +-----+ +-----+ +-----+ +-----+ |
|
| +-----v-----+ |
| | Sequelize ORM | |
| +-----+-----+ |
|
| +-----v-----+ |
| | PostgreSQL (10 tables) | |
| | utilisateurs, candidats, instituts, programmes, | |
| | candidatures, notifications, medias, favoris, | |
| | demandes_acces, pre_inscriptions | |
| +-----+-----+ |
|
| +-----+-----+ |
| | diplomaVerifierService.js -- client HTTP interne | |
| +-----+-----+ |
+-----+
|
| HTTP POST /api/verify
|
+-----v-----+
| DIPLOMA VERIFIER -- Python / FastAPI |
|
| +-----+-----+ |
| | Orchestrator V7 | |
|
| +-----+ +-----+ +-----+ +-----+ |
| | OCR | | Text | | Scoring | |Tampering | |
| |Tesseract| | Analyzer | | Engine | |Detector | |
| | 5 langs | | spaCy | | V7 Multi | | ELA+DCT | |
| +-----+ +-----+ +-----+ +-----+ |
|
|

```

	+-----+	+-----+		
	Signature	Stamp		
	Detector	Detector		
	(OpenCV)	(Hough)		
	+-----+	+-----+		
	+-----+			
	Reponse : { score: 0-100, confidence_level, reasons, v7{} }			
	+-----+			

2.2 Diagramme de base de donnees

+-----+	+-----+		
UTILISATEUR	--1:1-- CANDIDAT		
(auth + role)	(profil etud)		
	+-----+		
id (UUID)	candidat_id		
email	+-----+	+-----+	
mot_de_passe	CANDIDATURE	----- PROGRAMME	
role (ENUM)	(dossier)	(formation)	
est_actif			
+-----+	statut (ENUM)	titre	
	lettre_motiv.	niveau	
1:1	docs_soumis	documents	
	(JSONB)	_requis	
+-----+	+-----+	(JSONB)	
INSTITUT		+-----+	
(profil ecole)			
	+---- MEDIA		
nom, sigle	(fichier)		
logo, contact			
validation_	----- (1:N) -----+		
status (ENUM)			
+-----+			
--- NOTIFICATION (1:N Utilisateur)			
--- DEMANDE_ACCES (workflow auto-inscription)			
--- PRE_INSCRIPTION (post-acceptation)			
--- FAVORI (junction Candidat <-> Programme)			

3. FLUX DETAILLES END-TO-END

3.1 Flux d'authentification JWT

FRONTEND	BACKEND	DB
POST /api/auth/login		
{ email, password }		
+----->		
	findOne({ email })	
	+----->	
	<-----+	
	bcrypt.compare()	
	jwt.sign({ id, role })	
	<- token (7j)	
<-----+		
{ token, utilisateur,		
profil }		
localStorage.setItem(		

```

| 'auth_token', token) |
| AuthContext.setUser() |
|
| Prochaine requete : |
| Authorization: Bearer |
| <token> |
+----->|
|
| authMiddleware : |
| jwt.verify(token) |
| findByPk(decoded.id) |
| -> req.user = { |
|   id, role, |
|   candidat_id, |
|   institut_id } |

```

Pourquoi JWT ?

- Stateless : le serveur ne stocke pas de session, parfait pour l'architecture REST
- Scalable : n'importe quel serveur peut valider le token sans base de données partagée
- Payload utile : { id, role } évite une requête DB à chaque route (profil résolu dans authMiddleware)

3.2 Flux de candidature complet

CANDIDAT	BACKEND	DIPLOMA VERIFIER
1. Cree brouillon		
POST /api/candidatures		
+ fichiers (FormData)		
+----->		
	Multer : sauve fichiers	
	Cree Media en DB (ACID tx)	
	Cree Candidature (brouillon)	
<-----+		
{ candidature_id }		
2. Soumet candidature		
POST /candidatures/		
:id/soumettre		
+----->		
	[Transaction ACID ouverte]	
	A. Mise a jour profil	
	candidat (whitelist)	
	B. Verifier profil complet	
	(prenom, nom, tel, nat...)	
	C. Verifier docs obligatoires	
	(documents_requis prog)	
	D. Analyse diplome (async)	
	+----->	
	POST /api/verify	
	{ file: diplome.pdf }	
	<-----+	
	{ score: 78,	
	niveau: "high",	
	subscores: {...} }	
	Stocke dans notes_institut:	
	"[DiplomaVerifier] score=78"	
	E. statut -> "soumise"	
	F. Notifications (candidat	
	+ institut)	

```

|
| [Transaction ACID fermee]
|<-----+
| { candidature: {...} }|

```

Points clés :

1. La verification OCR est non-bloquante : si elle echoue, la candidature est quand meme soumise
2. La transaction ACID garantit que si une etape echoue, rien n'est persiste
3. Les notifications sont envoyees dans la meme transaction

3.3 Flux de verification de diplome (Diploma Verifier)

FICHER PDF/IMAGE

```

|
v
+-----+
| ETAPE 1 -- Conversion |
| image_converter.py : PDF -> PIL Image -> OpenCV (BGR) |
+-----+
|
v
+-----+
| ETAPE 2 -- OCR (5 passes paralleles) |
| Tesseract 5 : fra, eng, ara, ara+fra, fra+eng |
| Auto-rotation (deskew), upscale x2 |
| Selection du meilleur texte (score semantique ou longueur) |
+-----+
|
+-----+
| | |
v v v
+-----+ +-----+ +-----+
| text_ | |signature_ | | stamp_ |
| analyzer | | detector | | detector |
| (spaCy NER) | | (contours)| | (Hough) |
| NOM, DATE, | | | |
| INSTITUTION | | sig_conf | | stamp_conf |
+-----+ +-----+ +-----+
| | |
+-----+
|
v
+-----+
| ETAPE 3 -- Critical Fields Validator (V7) |
| Valide : prenom, nom, date, institution, specialisation |
| Score 0-100 par champ, agrege avec poids (CRITICAL_FIELDS_ |
| WEIGHTS). Detecte les templates officiels sans identite. |
+-----+
|
v
+-----+
| ETAPE 4 -- Tampering Detector (si diploma_confidence >= 0.2) |
| 5 composants : ELAxDCT, copy-paste, fond, EXIF, bruit |
| + MantraNet (6e composant, desactive par default) |
| -> fraud_score (0-100, INVERSE dans scoring) |
+-----+
|
v
+-----+
| ETAPE 5 -- Scoring Engine V7 Multi-Score |
|
| 6 sous-scores (0-100) :
| +-----+
| | Sous-score | Poids | Source |

```

```
| +-----+-----+-----+ |
| | structure      | 0.14 | structure_count + phrases | |
| | semantic       | 0.10 | spaCy + coherence_score  | |
| | critical_fields | 0.20 | CF Validator V7          | |
| | visual_auth    | 0.10 | sigx0.4 + stampx0.6     | |
| | fraud_trust    | 0.36 | 100 - fraud_score        | |
| | ocr_confidence  | 0.10 | Tesseract confidence     | |
| +-----+-----+-----+ |
|                               |
| + Caps de securite : no-content, hallucination, |
|   semantic ceiling, template, fraud hard cap  |
| +-----+-----+-----+ |
|
|
| v
| { score: 0-100, confidence_level,
|   reasons:[...], v7: { subscores, risks } }
```

3.4 Flux upload document

FRONTEND	BACKEND (Multer)	DISK
FormData {		
diplome: File,		
lettre_motivation: string		
}		
Content-Type: multipart/form-data		
+----->		
	diskStorage (./uploads/)	
	+----->	
	nom: timestamp-random	
	ext: .pdf/.jpg/.png	
	Media.create({	
	chemin: 'uploads/...',	
	type_mime,	
	taille_octets	
	})	
	Candidature.update({	
	documents_soumis: [	
	{ nom, url, media_id	
	telecharge_le }	
	]	
	}) // JSONB	

3.5 Flux workflow Institut (invitation -> validation)

ADMIN	BACKEND	EMAIL	INSTITUT	CANDIDATS
	Invite institut			
	POST /instituts			
+----->				
	first_login_			
	token (UUID)			
	Cree en DB			
	+----->			
	Email avec			
	lien one-use			
		+----->		
		Clique lien		
		/first-login		
	<-----+			
	POST /premier-			

	login/terminer		
	{ token, pwd,		
	nom, tel }		
	validation_		
	status ->		
	pending_admin_		
	review		
Valide l'institut			
POST /instituts/			
:id/approuver			
+----->			
	validation_		
	status ->		
	approved		
		Notif email	
	+----->	----->	
		Publie	
		programmes	
		+----->	
		Visible sur	
		la plateforme	

4. EXPLICATION DOSSIER PAR DOSSIER

4.1 BACKEND

backend/controllers/

Role : Couche HTTP mince -- recoit la requete, delegue au service, renvoie la reponse.

Pattern architectural : Thin Controller (controleur mince)

Principe : Un controller ne doit contenir que :

1. Extraction des parametres (req.body, req.params, req.user)
2. Appel du service metier
3. Formatage de la reponse HTTP

Exemple concret :

```
authController.js -> recoit POST /login -> delegue bcrypt + JWT -> renvoie token
candidatureController.js -> recoit POST /soumettre -> delegue a candidatureWorkflow.js
```

Communication avec les autres couches :

- Recoit depuis : routes/ (Express router)
- Delege vers : services/ (logique metier)
- Lit depuis : req.user (peuple par authMiddleware)

backend/services/

Role : Coeur metier -- logique decouplee des routes HTTP.

Pattern architectural : Service Layer (couche service)

Fichiers cles :

Fichier	Role
candidatureWorkflow.js	Machine a etats des candidatures

notificationService.js	Envoi des notifications en DB
emailService.js	SMTP Nodemailer (invitation + reset)
preInscriptionService.js	Pre-inscription post-acceptation
diplomaVerifierService.js	Client HTTP vers le microservice OCR

Pourquoi separer services et controllers ?

- Testabilite : on peut tester la logique sans lancer le serveur HTTP
- Reutilisabilite : plusieurs routes peuvent appeler le meme service
- Lisibilite : chaque fichier a une responsabilite claire

backend/middleware/

Role : Intercept les requetes HTTP AVANT le controller.

Fichiers :

Fichier	Role
authMiddleware.js	Verifie JWT, resout profil, peuple req.user
candidatureGuards.js	Verifie propriete du dossier + statut terminal
rateLimiter.js	Limite le nombre de requetes par IP
upload.js	Configure Multer (stockage disque, filtres MIME)

Analogie : Les middleware sont comme les controles de securite a l'aeroport -- vous passez obligatoirement avant d'accéder a la porte d'embarquement (le controller).

backend/models/

Role : Definit le schema de la base de donnees via Sequelize ORM.

Pattern architectural : Active Record via Sequelize

Structure :

```
models/index.js    <- charge tous les modeles + declare les associations
models/Utilisateur.js <- table utilisateurs
models/Candidat.js  <- profil etudiant
models/Institut.js  <- profil ecole
models/Programme.js <- formation
models/Candidature.js <- dossier (noeud central)
models/Notification.js <- alertes
models/Media.js      <- fichiers uploades (polymorphe)
models/Favori.js     <- junction candidat <-> programme
models/DemandeAcces.js <- demandes auto-inscription
models/PreInscription.js <- formulaire post-acceptation
```

Types remarquables utilises :

- UUID v4 : cles primaires non-devinables, pas de sequence entiere exposee
- JSONB : stockage flexible (documents_soumis, adresse, contact, parcours_academique)
- ENUM : valeurs fermees (role: candidat/institut/admin, statut candidature)
- ARRAY(STRING) : langues, accreditations

backend/migrations/

Role : Historique versionnee du schema de base de donnees.

Principe important :

Le schema BDD est UNIQUEMENT gere par les migrations. sequelize.sync() est INTERDIT en production.

Pourquoi ?

- sync() peut silencieusement modifier ou supprimer des colonnes
- Les migrations permettent de monter ET de descendre (undo) de version
- Tracabilite complete des changements de schema

backend/routes/

Role : Monte les controllers sur les chemins URL et applique les middleware.

```
/api/auth      -> authRoutes.js
/api/programmes -> programmeRoutes.js
/api/candidatures -> candidatureRoutes.js
...
```

backend/uploads/

Role : Stockage local des fichiers uploades par les candidats.

Servi statiquement via /uploads par Express.

4.2 FRONTEND

src/context/

Role : Etat global d'authentification partage dans toute l'application.

Pattern architectural : Context API + Provider Pattern (React)

AuthContext.tsx expose : user, token, isAuthenticated, login, logout, register, updateUser.

src/services/api.ts

Role : Client HTTP centralise -- toutes les requetes API passent par la.

Pattern : Service Layer + Singleton Axios

Ce qu'il fait :

1. Configure une instance Axios avec baseUrl, timeout, headers
2. Intercepteur requete : injecte automatiquement le JWT depuis localStorage
3. Intercepteur reponse : gere 401 (session expiree -> redirect /login) et 403
4. Expose 9 services types : authService, programmeService, candidatureService...

src/hooks/

Role : Data-fetching reutilisable avec etats loading, error, data.

Pattern architectural : Custom Hooks (React)

Chaque hook encapsule : l'appel API, le state management, le re-fetch.

```
usePrograms.ts      -> GET /api/programmes (avec filters + pagination)
useCandidatures.ts  -> GET /api/candidatures/mine
useFavoris.ts       -> GET /api/favoris/mine
useNotifications.ts -> GET /api/notifications/mine
```

src/app/pages/

Role : Pages de niveau route (une page = une URL).

23 routes incluant : Home, SearchResults, ProgramDetail, InstitutionProfile, Compare, Login, Signup, FirstLogin, ResetPassword, CandidateDashboard, MesCandidatures, MesFavoris, MesDocuments, Parametres, InstitutionDashboard, AdminDashboard, DemandeAcces, PreInscription.

src/app/components/

Role : Composants reutilisables organises par domaine.

```

admin/      -> sections du dashboard admin
institution/ -> sections du dashboard institut
forms/      -> composants de formulaires (NationaliteSelect...)
ui/         -> shadcn/ui (NE PAS EDITER)
figma/      -> artefacts Figma (NE PAS EDITER)
NotificationDropdown.tsx -> badge + dropdown temps reel

```

src/components/ProtectedRoute.tsx

Role : Garde les routes dashboard -- redirige si non authentifie ou mauvais role.

Pattern architectural : Route Guard (Higher-Order Component)

src/i18n/

Role : Internationalisation FR/EN via i18next.

Principe : Aucune chaine UI n'est ecrite en dur dans les composants -- toujours via t('cle').

src/styles/

Role : Design system custom.

```

tailwind.css -> classes Tailwind
theme.css    -> variables CSS (dark/light mode)
fonts.css    -> imports de polices
edubridge.css -> variables --edu-* (--edu-blue, --edu-danger...)

```

4.3 DIPLOMA VERIFIER

app/services/orchestrator.py

Role : Chef d'orchestre -- coordonne tous les services d'analyse dans l'ordre.

Pattern architectural : Orchestrator Pattern (monolithe modulaire)

app/services/ocr_service.py

Role : Extraction de texte via Tesseract -- 5 passes paralleles, selection du meilleur resultat.

app/services/text_analyzer.py

Role : Analyse semantique du texte extrait -- classification, coherence, NER spaCy.

app/services/signature_detector.py

Role : Detecte une signature manuscrite via analyse de contours OpenCV.

app/services/stamp_detector.py

Role : Detecte un cachet circulaire via transformee de Hough (cercles).

app/services/critical_fields_validator.py

Role : Valide les champs critiques d'un diplome (nom, prenom, date, institution, specialisation). Resout les faux positifs sur les templates officiels sans identite etudiante.

app/services/scoring_engine.py

Role : Moteur de scoring multi-score -- 6 sous-scores independants, caps de securite, score global 0-100.

app/services/tampering_detector.py

Role : Detection de falsification -- 5 composants (ELA, copy-paste, fond uniforme, EXIF, bruit) + MantraNet optionnel (6e).

app/config.py

Role : Source de verite unique pour tous les seuils, poids, mots-cles et constantes du microservice.

5. EXPLICATION FICHIER PAR FICHIER

FICHIER 1 : backend/services/candidatureWorkflow.js

Niveau de criticite : COEUR METIER

Role :

Moteur de workflow -- implemente la machine a etats des candidatures. C'est le fichier le plus important du backend.

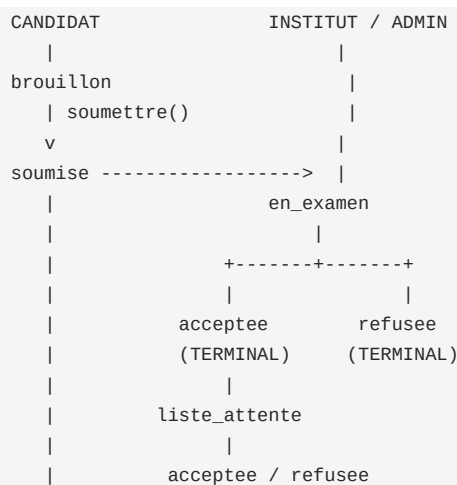
Pourquoi il est important :

Toute la logique metier des candidatures (transitions, validations, anti-doublon, OCR, notifications) est ici. C'est le seul endroit ou un candidat peut passer de brouillon a soumise, et ou un institut peut accepter ou refuser.

Fonctions principales :

Fonction	Role
creerBrouillon()	Cree une candidature avec statut initial
mettreAJourBrouillon()	Ajoute des documents ou modifie la lettre
soumettre()	Pipeline en 5 etapes : profil -> docs -> OCR -> statut
changerStatut()	Transition par l'institut ou l'admin
validerTransition()	Verifie si la transition est autorisee pour le role
verifierCompleture()	Compare docs soumis vs docs_requis du programme
verifierProfilCompleet()	Verifie les champs obligatoires du candidat
verifierDoublon()	Anti-doublon (un candidat = une candidature par program

Machine a etats :



Flux des donnees dans soumettre() :

```

req.body -> whitelist CHAMPS_PROFIL_AUTORISES
        -> candidat.update() [transaction]
        -> verifierProfilCompleet() [sync]
        -> verifierCompleture() [async, DB]

```

```
-> verifierDiplome() [async, HTTP]
-> candidature.update({ statut: 'soumise' })
-> notif candidat + notif institut
```

Pattern anti-mass assignment (ligne 9-16) :

La whitelist CHAMPS_PROFIL_AUTORISEES empeche qu'un candidat malveillant envoie un champ comme role: 'admin' dans le body et qu'il soit persiste.

Ce que le jury peut demander :

- "Pourquoi une transaction ACID ici ?"
- "Que se passe-t-il si la verification OCR echoue ?"
- "Comment gerez-vous les statuts terminaux ?"

Reponses :

- Transaction ACID : si la notification echoue apres le changement de statut, le rollback remet le statut precedent -- coherence garantie.
- OCR non-bloquant : encapsule dans try/catch, la soumission reussit meme si le microservice est down.
- Statuts terminaux ['acceptee', 'refusee'] : validerTransition() leve une exception 400 immediatement.

FICHER 2 : backend/middleware/authMiddleware.js

Niveau de criticite : ELEVE

Role :

Verificateur de JWT -- peuple req.user avec les informations d'identite pour toutes les routes protegees.

Ce qu'il fait ligne par ligne :

```
// 1. Extraire le token du header Authorization
const token = authHeader.split(' ')[1]; // "Bearer <token>"

// 2. Verifier cryptographiquement la signature JWT
const decoded = jwt.verify(token, process.env.JWT_SECRET);
// -> Lance TokenExpiredError ou JsonWebTokenError si invalide

// 3. Charger l'utilisateur depuis la DB (verifier qu'il existe encore)
const utilisateur = await Utilisateur.findPk(decoded.id);

// 4. Verifier que le compte est actif
if (!utilisateur.est_actif) { return 403 }

// 5. Resoudre le profil (institut_id ou candidat_id)
// -> Necessaire pour les guards de propriete (un institut ne peut
//     agir que sur SES programmes et candidatures)
```

4 fonctions exportees :

Fonction	Usage
authMiddleware	Route protegee standard (candidat, institut, admin)
isAdmin	Reserve admin -- a chainer apres authMiddleware
restrictTo(...roles)	Restreint a une liste de roles
optionalAuth	Route publique avec conscience du role (ex: listing ins)

Pourquoi verifier est_actif en DB a chaque requete ?

Le JWT peut avoir une duree de 7 jours. Si un compte est suspendu, le JWT existant resterait valide pendant 7 jours sans cette verification DB. Le cout (1 requete DB par requete) est acceptable pour la securite.

FICHIER 3 : backend/controllers/authController.js

Niveau de criticite : ELEVE

Role :

Gestion de l'authentification -- inscription, connexion, first login, reset password.

Fonctions principales :

Fonction	Route	Description
register	POST /auth/register	Inscription candidat uniquement (transaction)
login	POST /auth/login	Connexion + verification suspension
validerTokenPremierLogin	GET /premier-login/valider	Valide le lien d'invitation
terminerPremierLogin	POST /premier-login/terminer	Active le compte institut
demanderResetPassword	POST /mot-de-passe/oublie	Genere token reset (anti-enumeration)
validerResetToken	GET /mot-de-passe/valider-token	Verifie expiration
reinitialiserPassword	POST /mot-de-passe/reinitialiser	Applique le nouveau mot de passe

Securite -- points importants :

1. Anti-enumeration (ligne 317-339) : La reponse au reset password est identique que l'email existe ou non ("Si un compte existe...") -- empeche de deviner les comptes.
2. Token reset securise (ligne 343) : crypto.randomBytes(32).toString('hex') -- 256 bits d'entropie, impossible a brute-forcer.
3. Bcrypt 10 rounds (ligne 55) : Hachage lent volontairement -- protege contre les attaques par dictionnaire si la DB est compromise.
4. Validation mot de passe fort (ligne 223) : Regex /^(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#\$%^&]).{8,}\$/ -- majuscule + chiffre + special + 8 chars minimum.
5. Transaction atomique register (ligne 57-73) : Utilisateur + Candidat crees ensemble -- si une operation echoue, rien n'est persiste.

FICHIER 4 : backend/models/index.js

Niveau de criticite : ELEVE

Role :

Point d'entree des modeles -- charge les 10 modeles et declare toutes les associations Sequelize.

Pattern : Factory Function (chaque modele est une fonction qui recoit sequelize + DataTypes)

Associations declarees :

```

Utilisateur 1:1 Candidat    (CASCADE delete)
Utilisateur 1:1 Institut    (CASCADE delete)
Institut 1:N Programme      (CASCADE delete)
Candidat N:N Programme      via Candidature
Candidature 1:N Media        (polymorphique)
Utilisateur 1:N Notification
Candidat N:N Programme      via Favori
Candidature 1:1 PreInscription

```

Polymorphisme Media (lignes 47-68) :

Un Media peut appartenir a un Candidat OU a un Institut. L'association constraints: false est necessaire car Sequelize ne peut

pas creer une vraie FK vers deux tables differentes. Le discriminant type_proprietaire identifie le type.

FICHIER 5 : frontend/src/context/AuthContext.tsx

Niveau de criticite : ELEVE

Role :

Etat global d'authentification -- persiste le JWT, expose les fonctions login/logout, valide silencieusement la session au demarrage.

Comportement au demarrage (useEffect ligne 59-106) :

```
1. Lire token + user depuis localStorage
2. Si absents -> loading=false, fin
3. Affichage optimiste : setUser(storedUser) immediatement (pas d'ecran blanc)
4. GET /auth/me en arriere-plan (validation silencieuse)
   -> Si OK : rafraichir le profil (validation_status peut avoir change)
   -> Si KO : logout (token expire ou revoke)
```

Pourquoi l'affichage optimiste ?

Sans ca, a chaque rechargement de page, l'utilisateur verrait un spinner pendant la validation. L'affichage optimiste evite ce flash d'interface.

construireUser() (lignes 20-45) :

Normalise la reponse backend en un objet User type cote frontend. Centralise la logique de mapping profil.id -> candidat_id ou -> institut_id.

FICHIER 6 : frontend/src/services/api.ts

Niveau de criticite : ELEVE

Role :

Client HTTP centralise -- toutes les requetes API passent obligatoirement ici.

Intercepteur requete (lignes 27-37) :

```
// Auto-injection du JWT
const token = localStorage.getItem('auth_token');
if (token) config.headers.set('Authorization', `Bearer ${token}`);

// FormData : supprimer Content-Type pour laisser le navigateur
// ajouter le boundary multipart automatiquement
if (config.data instanceof FormData) {
  config.headers.delete('Content-Type');
}
```

Intercepteur reponse (lignes 40-57) :

```
// 401 avec token present = session expiree -> logout
if (status === 401 && localStorage.getItem('auth_token')) {
  // -> redirect /login
}
// 403 -> redirect /
```

Pourquoi ne pas faire de fetch() direct dans les pages ?

1. Duplication du code d'injection JWT dans chaque composant
2. Gestion d'erreur inconsistante
3. Impossible de changer le baseUrl en un seul endroit
4. Testabilite difficile

FICHIER 7 : frontend/src/components/ProtectedRoute.tsx

Niveau de criticite : MOYEN

Role :

Garde les routes dashboard -- trois niveaux de protection :

1. Loading -> spinner
2. Non authentifie -> redirect /login
3. Mauvais role -> redirect /
4. Institut suspendu -> ecran plein blocage
5. OK -> rend les enfants

Ce que le jury peut demander :

"Est-ce suffisant comme securite cote frontend ?"

Reponse :

Non, et c'est voulu. La vraie securite est cote backend (JWT + authMiddleware + restrictTo). Le ProtectedRoute est juste de l'UX -- il evite d'afficher une page vide si le backend renvoie 403. Un utilisateur malveillant peut bypasser le frontend, mais pas le backend.

FICHIER 8 : diploma-verifier/app/services/orchestrator.py

Niveau de criticite : COEUR METIER

Role :

Chef d'orchestre du pipeline d'analyse -- 12 etapes sequentielles.

Gestion des cas limites :

Cas	Comportement
Document vraiment vide (pas de texte, pas de signature,	Early return score 5-18 (deterministe)
Pas un diplome (diploma_confidence < 0.2) ET pas de sig	Early return score 5-25
Erreur fatale	Score fallback 5-20 (deterministe, base sur hash filena
Tampering detector plante	Graceful degradation, fraud_score=0, pipeline continue

Determinisme :

Meme fichier -> meme score toujours. Utilisation de SHA-256 comme graine de Random pour eviter la variabilite non-maitrisee.

FICHIER 9 : diploma-verifier/app/services/scoring_engine.py

Niveau de criticite : COEUR METIER

Role :

Moteur de scoring -- 6 sous-scores + caps de securite + score global.

Architecture V7 Phase 2 :

```
compute_subscores()
|
v SubscoreBundle {
    structure_score:      0-100
    semantic_score:       0-100
    critical_fields_score: 0-100
    visual_authenticity_score: 0-100
    fraud_score:          0-100 (INVERSE)
```

```

        ocr_confidence_score:      0-100
    }
    |
    v
compute_global_trust_score()
1. Poids dynamiques (si OCR faible -> moins de poids texte, plus visuel)
2. Somme ponderee (fraud_trust = 100 - fraud_score)
3. Caps de securite (6 caps)
4. Clamp [3, 98]
    |
    v
compute_risk_level() : trusted | review_recommended | suspicious | highly_suspicious
    |
    v
risk_level_to_confidence() : high | medium | low | very_low

```

Pourquoi le score ne va jamais a 0 ni a 100 ?

- 0 = certitude absolue de faux -> impossible en heuristique
- 100 = certitude absolue d'authentique -> impossible sans verification humaine
- Plage [3, 98] : honnetete epistemique

Fraud_trust (ligne 970) :

`fraud_trust = 100 - fraud_score`

Le `fraud_score` est INVERSE dans la somme ponderee car plus il est eleve, moins on fait confiance au document. Cette inversion permet de l'integrer naturellement dans la somme positive.

6. ANALYSE DU CODE LIGNE PAR LIGNE

6.1 candidatureWorkflow.js -- soumettre() -- Analyse detaillee

```

// == ETAPE 1 : Verifier la transition =====
validerTransition('candidat', candidature.statut, 'soumise');
// -> Leve une exception si statut est terminal ('acceptee', 'refusee')
// -> Verifie que brouillon -> soumise est autorise pour le role 'candidat'
// -> Un candidat ne peut PAS aller de 'soumise' a autre chose

// == ETAPE 2 : Transaction ACID =====
return sequelize.transaction(async (t) => {
  // -> Toutes les operations suivantes sont atomiques
  // -> Si une echoue -> rollback automatique de tout

  // -- A. Auto-update profil avec whitelist -----
  const maj = {};
  for (const champ of CHAMPS_PROFIL_AUTORISES) {
    if (profil[champ] !== undefined) maj[champ] = profil[champ];
  }
  // -> Ignore silencieusement 'role', 'est_actif', 'email'...
  // -> Securite : mass-assignment impossible

  await candidat.update(maj, { transaction: t });
  // Le hook beforeValidate du modele Candidat sera declenche
  // (validation CIN/passeport selon nationalite)

  // -- B. Verification profil -----
  const profilCheck = verifierProfilCompleet(candidat);
  if (!profilCheck.complet) {
    throw { status: 400, manquants_profil: profilCheck.manquants };
    // -> Le frontend peut afficher precisement quels champs manquent
  }
}

```

```
// -- C. Verification documentaire -----
const docCheck = await verifierCompleture(candidature);
// -> Compare documents_soumis (JSONB) vs documents_requis du programme
// -> Lookup O(1) via Set (optimisation)
if (!docCheck.complet) {
  throw { status: 400, manquants: docCheck.manquants };
}

// -- D. OCR non-bloquant -----
const resultatVerif = await verifierDiplome(cheminFichier, docDiplome.nom);
// -> Si le microservice est down -> resultatVerif.succes = false
// -> La soumission continue quand meme (non-bloquant !)
// -> Le score est stocke dans notes_institut pour l'instituteur

// -- E. Changement de statut + notifications -----
await candidature.update({ statut: 'soumise' }, { transaction: t });
await notif.notifierChangementStatut(candidature, ancien_statut, 'soumise', t);
await notif.notifierNouvelleCandidature(candidature, t);
// -> Les notifications sont dans la MEME transaction
// -> Si la notif echoue -> rollback + le statut n'est PAS change
});
```

6.2 authMiddleware.js -- Analyse complete

```
const authMiddleware = async (req, res, next) => {
  // 1. Extraction du token
  const authHeader = req.headers.authorization;
  // Format attendu : "Bearer eyJhbGci..."
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Token manquant...' });
  }

  const token = authHeader.split(' ')[1];

  // 2. Verification cryptographique
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  // -> Si la signature est invalide -> JsonWebTokenError -> catch -> 401
  // -> Si expire -> TokenExpiredError -> catch -> 401
  // -> Si OK -> decoded = { id: 'uuid', role: 'candidat', iat, exp }

  // 3. Verification DB (compte toujours actif ?)
  const utilisateur = await Utilisateur.findByPk(decoded.id, {
    attributes: ['id', 'role', 'est_actif', 'first_login_completed'],
    // -> attributes limites : on ne charge PAS le mot de passe
  });

  // 4. Resolution du profil lie
  if (utilisateur.role === 'institut') {
    const institut = await Institut.findOne({
      where: { utilisateur_id: utilisateur.id },
      attributes: ['id', 'validation_status', 'suspension_reason'],
    });

    if (institut?.validation_status === 'suspended') {
      return res.status(403).json({
        code: 'ACCOUNT_SUSPENDED',
        reason: institut.suspension_reason,
        // -> Le frontend affiche l'ecran de suspension avec le motif
      });
    }
    user.institut_id = institut.id;
    // -> Utilise dans les guards pour verifier ownership
  }
}
```

```
req.user = user;
next(); // -> Passe au middleware ou controller suivant
};
```

6.3 scoring_engine.py -- compute_global_trust_score() -- Analyse

```
def compute_global_trust_score(subscores, ...):
    # 1. Poids dynamiques selon qualite OCR
    weights = _compute_dynamic_weights_v7(subscores)
    # Si ocr_confidence_score < 50 :
    # -> reduit le poids semantic (-40%)
    # -> reduit le poids structure (-30%)
    # -> augmente le poids visual_authenticity (+5%)
    # Raison : si l'OCR a rate, les signaux textuels sont peu fiables

    # 2. Inversion fraud (INVERSE dans la somme)
    fraud_trust = 100 - subscores.fraud_score
    # fraud_score = 80 (tres suspect) -> fraud_trust = 20 (penalise fort)
    # fraud_score = 0 (clean) -> fraud_trust = 100 (bonus maximum)

    # 3. Somme ponderee
    raw = sum(weights[k] * values[k] for k in weights)
    # Exemple document authentique typique :
    # structure=85 x 0.14 = 11.9
    # semantic=70 x 0.10 = 7.0
    # cf_score=90 x 0.20 = 18.0
    # visual=80 x 0.10 = 8.0
    # fraud_trust=95 x 0.36 = 34.2
    # ocr_conf=80 x 0.10 = 8.0
    # TOTAL = 87.1 -> score "trusted"

    # 4. Caps de securite
    # -> no_content (structure=0 ET semantique<10) -> cap 18
    # -> hallucination (texte court + pas de degree/date) -> cap 18
    # -> semantic_ceiling (semantique<15 ET score>70) -> cap 70
    # -> template (visual>80 ET CF<40) -> cap 50
    # -> fraud_hard_cap (fraud>80) -> cap 35

    # 5. Clamp final [3, 98]
    final = int(round(max(V7_SCORE_MIN, min(V7_SCORE_MAX, raw))))
```

6.4 AuthContext.tsx -- Affichage optimiste + validation silencieuse

```
useEffect(() => {
    // Etape 1 : Lecture du cache localStorage
    const storedToken = localStorage.getItem('auth_token');
    const storedUser = lireUtilisateurStocque();

    if (!storedToken || !storedUser) {
        setLoading(false); // -> Pas de session -> rendu immediat
        return;
    }

    // Etape 2 : AFFICHAGE OPTIMISTE
    // On affiche l'UI immediatement sans attendre la validation reseau
    setToken(storedToken);
    setUser(storedUser);
    setLoading(false); // <- loading=false AVANT la requete reseau !
    // -> L'utilisateur voit son dashboard immediatement

    // Etape 3 : Validation silencieuse en arriere-plan
    axios.get('/auth/me', { headers: { Authorization: `Bearer ${storedToken}` } })
        .then(({ data }) => {
            // Rafraichir le profil (ex: validation_status peut avoir change)
```

```

const freshedUser = construireUser(data.utilisateur, profil);
localStorage.setItem('auth_user', JSON.stringify(freshedUser));
setUser(freshedUser); // -> UI mise a jour silencieusement
})
.catch(() => {
  // Token expire ou revoke -> logout propre
  localStorage.removeItem('auth_token');
  localStorage.removeItem('auth_user');
  setToken(null);
  setUser(null);
  // -> ProtectedRoute detecte isAuthenticated=false -> redirect /login
});
}, []);

```

7. QUESTIONS PROBABLES DU JURY -- AVEC REPONSES

7.1 Questions sur l'architecture

Q : Pourquoi avoir choisi une architecture monolithique avec microservice au lieu d'une architecture microservices complete ?

Reponse courte : Le perimetre fonctionnel du projet ne justifiait pas la complexite operationnelle des microservices. Le diploma-verifier est naturellement isole car il est stateless, CPU-intensif, et peut etre scale independamment.

Reponse longue soutenance :

"Nous avons adopte une architecture hybride : un backend monolithique pour le coeur metier (ou les entites sont fortement couplees -- candidature, programme, institution, notification), et un microservice isole pour la verification documentaire. Ce choix est justifie par plusieurs raisons : le microservice est stateless (pas de DB, pas d'authentification), il peut etre scale horizontalement independamment si la charge OCR augmente, et il utilise un stack technologique different (Python/FastAPI) plus adapte au traitement d'images. Une architecture microservices complete aurait introduit de la complexite reseau inutile pour des entites qui doivent etre coherentes dans la meme transaction ACID."

Q : Comment garantisiez-vous la coherence des donnees lors d'une soumission de candidature ?

Reponse :

"Via les transactions ACID de Sequelize sur PostgreSQL. Dans soumettre(), la mise a jour du profil candidat, la verification documentaire, la verification OCR (stockage du score), le changement de statut et les notifications sont tous dans la meme transaction. Si une etape echoue, PostgreSQL fait un rollback automatique. La seule exception est la verification OCR qui est non-bloquante : si le microservice est indisponible, on log un warning mais la soumission continue -- la plateforme ne doit pas dependre de la disponibilite du microservice pour sa fonction principale."

Q : Pourquoi PostgreSQL et non MongoDB ?

Reponse :

"PostgreSQL a ete choisi pour plusieurs raisons. D'abord, nos donnees sont fortement relationnelles : candidats, programmes, instituts, candidatures forment un graphe d'entites avec des contraintes d'integrite importantes (FK, CASCADE). Ensuite, nous utilisons les transactions ACID pour le workflow de candidature -- MongoDB en mode standalone n'offrait pas les transactions multi-documents avec la meme robustesse. Enfin, PostgreSQL supporte le type JSONB qui nous permet d'avoir des champs flexibles (documents_soumis, adresse, contact) sans sacrifier la performance -- les colonnes JSONB sont indexables."

Q : Quelle est la difference entre un middleware et un controller ?

Reponse :

"Un middleware est une fonction qui s'exécute avant le controller et qui peut modifier la requête, renvoyer une réponse anticipée ou passer la main. Il implémente des préoccupations transversales : authentification (authMiddleware), contrôle d'accès (isAdmin, restrictTo), rate limiting, upload de fichiers. Un controller gère UNE ressource spécifique : il extrait les paramètres, appelle le service métier, et formate la réponse. La règle est que le controller doit être aussi mince que possible -- la logique métier est dans les services."

7.2 Questions sur la sécurité

Q : Comment protégez-vous l'API contre les attaques par force brute ?

Reponse :

"Deux niveaux de rate limiting via express-rate-limit : un limiteur global de 600 requêtes par 15 minutes par IP sur toutes les routes /api/*, et un limiteur strict de 20 requêtes par 15 minutes sur /api/auth/login avec skipSuccessfulRequests: true (une connexion réussie ne compte pas). Si la limite est dépassée, le serveur répond avec 429 et un message JSON standardisé. Ces limites sont surchargeables via variables d'environnement pour le déploiement."

Q : Comment stockez-vous les mots de passe ?

Reponse :

"Nous utilisons bcrypt avec 10 rounds de salt. Bcrypt est un algorithme de hachage adaptatif -- le coût computationnel peut être augmenté au fil du temps. À 10 rounds, le hachage prend environ 100ms sur un serveur moderne, ce qui rend les attaques par dictionnaire prohibitivement lentes. Le mot de passe n'est jamais retourné dans les réponses API : nous utilisons l'attribut exclude: ['mot_de_passe'] dans les queries Sequelize. Le token JWT ne contient que { id, role } -- jamais le mot de passe ni d'information sensible."

Q : Comment prévenez-vous les attaques par injection SQL ?

Reponse :

"Sequelize ORM génère des requêtes paramétrées (prepared statements) automatiquement. Toutes nos interactions avec PostgreSQL passent par Sequelize -- nous n'écrivons jamais de SQL brut avec concaténation de chaînes. Pour les recherches textuelles comme le filtre ?search=, nous utilisons Op.iLike de Sequelize qui échappe correctement les caractères spéciaux. L'upload de fichiers est géré par Multer avec validation du type MIME et une limite de taille de 5 Mo."

Q : Que se passe-t-il si le JWT est volé ?

Reponse :

"C'est la limitation classique des JWT stateless. Notre mitigation est la durée de vie courte (7 jours) et la vérification du statut est_actif en base de données à chaque requête -- si un compte est désactivé par un admin, le JWT devient immédiatement invalide même s'il n'est pas expiré. La colonne jeton_rafraichissement existe dans le schéma pour un futur refresh token flow qui permettrait des JWT de courte durée (15 minutes) avec rotation des tokens. Ce n'est pas implémenté dans le MVP par contrainte de temps."

Q : Qu'est-ce que le mass-assignment et comment le prévenez-vous ?

Reponse :

"Le mass-assignment est une vulnérabilité où un attaquant envoie des champs supplémentaires dans un body JSON et ceux-ci sont persistés sans validation. Par exemple, un candidat pourrait envoyer { role: 'admin' } dans le body de soumission et, sans protection, le serveur le persisterait. Nous l'évitions via la whitelist explicite

CHAMPS_PROFIL_AUTORISES dans candidatureWorkflow.js : seuls les champs declares dans cette liste peuvent etre mis a jour par un candidat. Tout autre champ est ignore silencieusement."

7.3 Questions sur les performances

Q : Quels sont les problemes de performance potentiels de votre backend ?

Reponse honnete :

"Nous avons identifie plusieurs points d'amelioration. D'abord, un probleme potentiel de N+1 queries dans GET /api/instituts : pour chaque institut, une requete separee charge ses programmes. La solution est d'utiliser separate: true ou include avec limit dans Sequelize. Ensuite, la pagination est appliquee sur les listings principaux (programmes, instituts, candidatures admin) avec un maximum de 100 par page, mais pas encore sur toutes les routes. L'OCR est le goulot d'etranglement majeur : une analyse peut prendre 2-5 secondes en fonction de la taille du fichier -- c'est pourquoi la verification est non-bloquante dans le workflow."

Q : Pourquoi 5 passes OCR en parallele ?

Reponse :

"Tesseract a des performances tres variables selon la langue du document et la combinaison de langues utilisee. Sur un diplome tunisien, le francais et l'arabe peuvent coexister. Une seule passe avec fra+eng+ara peut faire baisser la qualite de reconnaissance de chaque langue individuellement. Nous executons 5 configurations en parallele via ThreadPoolExecutor : ara, fra, eng, ara+fra, fra+eng. Pour chaque resultat, nous calculons un score semantique (nombre de mots-cles de diplome reconnus). Le resultat avec le meilleur score semantique -- ou le plus long texte en cas d'egalite -- est retenu."

7.4 Questions sur l'OCR et les heuristiques

Q : Pourquoi des heuristiques et non un modele de deep learning pour la verification ?

Reponse longue soutenance :

"Plusieurs raisons justifient ce choix. Premierement, un modele de classification supervisee necessite un corpus d'entrainement etiquete de diplomes authentiques et falsifies tunisiens -- nous n'en disposons pas. Les modeles pre-entraines comme MantraNet sont entraines sur des datasets comme CASIA, qui contient principalement des images de nature, pas des documents administratifs tunisiens. Deuxiemement, les heuristiques sont explicables : chaque sous-score (structure, semantic, critical_fields, visuel, tampering) a une interpretation metier claire. Un jury academique ou un institut peuvent comprendre pourquoi un document a obtenu un score de 45. Un reseau de neurones serait une boite noire. Troisiemement, notre pipeline deterministe garantit que le meme fichier produit toujours le meme score -- pas de variance due au non-determinisme GPU."

Q : Quelles sont les limites de votre systeme OCR ?

Reponse honnete :

"Plusieurs limites connues. Premierement, l'arabe manuscrit : Tesseract gere bien l'arabe imprime mais mal l'arabe manuscrit ou les calligraphies stylees. Deuxiemement, la qualite du scan : des diplomes photographies avec un smartphone en mauvaise lumiere ou avec un angle prononce produisent des resultats OCR degrades. Nous avons mis en place une auto-rotation pour corriger les legeres inclinaisons, mais pas les transformations perspectives complexes. Troisiemement, la divergence Tesseract 5.4 (Windows) vs 5.5 (Docker Linux) : les versions produisent des outputs legerement differents pour certains caracteres arabes. Nous avons des caps anti-hallucination dans le scoring engine pour attenuer ces variations."

Q : Qu'est-ce que l'ELA (Error Level Analysis) et comment l'utilisez-vous ?

Reponse :

"L'ELA est une technique de detection de manipulation d'images basee sur les artefacts de compression JPEG. Quand une image JPEG est re-compressee, les zones non modifiees montrent un niveau d'erreur uniforme, tandis que les zones qui ont ete copiees-collees ou modifiees depuis une source avec un niveau de compression different montrent des niveaux d'erreur anormalement eleves ou bas. Dans notre `tampering_detector.py`, nous re-compressons l'image a 90% de qualite JPEG et calculons la difference absolue avec l'original. Les regions avec un niveau d'erreur > seuil sont marquees comme suspectes. Combine avec la transformee DCT, cela constitue notre premier composant de detection de falsification."

Q : Pourquoi le `fraud_trust` a-t-il le poids le plus eleve (0.36) dans le score global ?

Reponse :

"C'est un choix delibere de politique de risque. Sur une plateforme academique, un faux diplome est plus prejudiciable qu'un vrai diplome mal score. Autrement dit : un faux negatif (accepter un faux) est plus couteux qu'un faux positif (rejeter un vrai pour re-verification manuelle). En donnant 36% du score a l'absence de tampering, nous nous assurons que meme un document structurellement parfait sera fortement penalise s'il presente des signes de manipulation. Le score global ne peut pas depasser 35 si `fraud_score > 80`, via le cap `fraud_hard_cap`."

7.5 Questions sur React / Frontend

Q : Pourquoi Context API et non Redux pour la gestion d'etat ?

Reponse :

"Le Context API est suffisant pour notre besoin : un etat global simple (utilisateur authentifie, token) partage dans toute l'application. Redux introduit de la complexite (actions, reducers, middleware) qui se justifie pour des etats complexes avec de nombreuses transformations. Pour une plateforme de notre taille, Context API offre une solution plus legere et nativement supportee par React. Pour des besoins de caching cote client (eviter les re-fetches), TanStack Query serait une evolution naturelle -- c'est documente dans nos TODOs."

Q : Pourquoi Vite plutot que Create React App ?

Reponse :

"Vite utilise ES Modules natifs en developpement -- le serveur de dev ne bundle pas le code, il le sert directement au navigateur. La difference de performance est significative : demarrage en < 500ms contre 10-30s pour CRA sur les gros projets. En production, Vite utilise Rollup pour le bundling optimise. Tailwind CSS v4 utilise egalement un plugin Vite natif qui integre le scanning CSS directement dans le pipeline de build, sans etape PostCSS separee."

Q : Qu'est-ce qu'un Custom Hook et pourquoi en avez-vous cree autant ?

Reponse :

"Un Custom Hook est une fonction dont le nom commence par `use` et qui peut appeler d'autres hooks React. Nous en avons cree un par ressource metier : `usePrograms`, `useCandidatures`, `useFavoris`, etc. Chaque hook encapsule le cycle de vie d'une requete API : etat initial loading, appel API, mise a jour de l'etat, gestion d'erreur, fonction de re-fetch. Sans ces hooks, chaque page devrait re-implementer ce pattern -- c'est de la duplication. Avec les hooks, une page n'a qu'a appeler `const { programmes, loading, error } = usePrograms(filters)` et l'UI se met a jour automatiquement."

7.6 Questions sur Docker et deployment

Q : Pourquoi Docker uniquement pour le diploma-verifier ?

Reponse :

"Le diploma-verifier a des dependances systeme complexes : Tesseract OCR 5.x, OpenCV, les modeles spaCy (fr_core_news_sm, xx_ent_wiki_sm), et potentiellement PyTorch pour MantraNet. Ces dependances sont difficiles a installer de maniere reproductible sur differentes plateformes (Windows vs Linux). Docker garantit que l'environnement est identique en developpement, test et production. Le backend Node.js et le frontend React ont des dependances purement npm -- facilement gerables sans conteneur. C'est un choix pragmatique : on conteneurise ce qui en a besoin."

7.7 Questions sur les limitations

Q : Quelles seraient vos priorites d'amelioration si vous aviez 3 mois supplementaires ?

Reponse :

"Trois priorites. Premierement, les tests automatises : zero tests actuellement -- nous ferions Jest + Supertest cote backend pour le workflow de candidature, et React Testing Library cote frontend pour les composants critiques. Deuxiemement, le refresh token flow : les JWT expirent en 7 jours, une colonne jeton_rafraichissement existe mais n'est pas utilisee -- implementer la rotation de tokens avec des access tokens de 15 minutes. Troisiemement, calibrer MantraNet : obtenir des poids valides sur un corpus de diplomes tunisiens et activer le 6e composant de tampering detection. Actuellement MantraNet est desactive (MANTRANET_ENABLED=False) car nous n'avons pas de corpus de calibration."

Q : Votre systeme peut-il detecter un faux diplome genere par IA ?

Reponse honnete :

"Partiellement. Le critical_fields_validator V7 a ete concu specifiquement pour ce cas : un diplome genere par IA a souvent une structure parfaite (tous les champs presents, cachet, signature) mais manque d'une identite etudiante reelle -- pas de nom de candidat specifique, pas de date precise, pas de specialisation. Le cap 'template' dans le scoring engine detecte ce pattern (visual_authenticity > 80 ET critical_fields < 40) et plafonne le score a 50. Cependant, un diplome IA avec une vraie identite injectee serait tres difficile a detecter sans un corpus d'entrainement. C'est pourquoi MantraNet (detection pixel-level) est prevu mais pas encore calibre."

8. FORCES ET FAIBLESSES DU PROJET

8.1 Forces

Architecture et design

Force	Explication
Separation des responsabilites	Controlleur mince / Service metier / Modele -- chaque co
Transactions ACID	Workflow de candidature atomique -- coherence garantie
Whitelist anti-mass-assignment	Securite defensive sur les mises a jour de profil
Microservice isole	OCR stateless, scalable independamment, en Python natif
Determinisme scoring	Meme fichier -> meme score toujours (hash-based noise)

Securite

Force	Explication
JWT + verification DB	Revocation immediate possible meme sans expiration token
Bcrypt 10 rounds	Protection contre vol de DB + dictionnaires
Rate limiting 2 niveaux	Anti-brute-force login + protection globale API
Anti-enumeration reset	Reponse identique email connu ou inconnu
ProtectedRoute + authMiddleware	Double securite (UX + API)
optionalAuth	Routes publiques avec conscience du role sans blocage

UX et fonctionnalites

Force	Explication
Affichage optimiste	Pas d'ecran blanc au rechargement
Bilingue FR/EN	i18next + detection automatique de langue
Scoring explicable	6 sous-scores + raisons humaines (pas de boite noire)
Non-bloquant OCR	La soumission reussit meme si le verifier est down
Workflow institut complet	Invitation email -> first login -> validation admin ->

8.2 Faiblesses honnetes

Techniques

Faiblesse	Degre	Mitigation actuelle / Plan
Zero tests automatises	Eleve	Validation manuelle via Swagger + scripts c
Pas de refresh token	Moyen	JWT 7j acceptable MVP, colonne presente p
N+1 queries instituts	Faible	Pagination limite le volume, separate: true p
Logs non structures	Faible	console.log + logger Python -- Winston prev
Pas d'audit trail	Moyen	cree_le/mis_a_jour_le presents, table Auditl

OCR

Faiblesse	Explication	Defense
Arabe manuscrit	Tesseract LSTM rate sur calligraphies	Score faible -> verification manuelle declen
Qualite scan	Smartphone = bruit -> OCR degrade	Caps anti-hallucination dans scoring engine
MantraNet desactive	Pas de poids calibres sur corpus tunisien	Architecture prete, plan de calibration docu
Tesseract 5.4 vs 5.5	Legere divergence Windows/Docker	Attenuuee par normalisation du texte avant h

8.3 Comment defendre ces choix intelligemment

Pour les tests manquants :

"Nous avons priorise l'implementation fonctionnelle pour le MVP. L'architecture est concue pour etre testable : les services sont decouplés des controllers, le workflow de candidature est une fonction pure testable sans HTTP. Les tests seraient la premiere priorite d'une phase suivante."

Pour MantraNet desactive :

"Activer un modele de deep learning sans corpus de calibration serait pire que de ne pas l'activer -- des faux positifs systematiques nuiraient a la confiance des candidats. Le choix de desactiver par default (MANTRANET_ENABLED=False) est une decision de qualite, pas un abandon."

Pour l'absence de refresh token :

"Un JWT de 7 jours est acceptable pour un MVP ou l'utilisateur se reconnecte regulierement. La suspension de compte via est_actif=false offre une revocation immediate qui compense l'absence de rotation automatique."

9. VOCABULAIRE SOUTENANCE

Termes techniques a maitriser

Terme	Definition simple	Definition technique
JWT	Badge numerique d'identite	JSON Web Token signe HMAC-SHA256, pay
ACID	Garantie de coherence BDD	Atomicite, Coherence, Isolation, Durabilite --
OCR	Extraction de texte d'une image	Optical Character Recognition -- Tesseract L
REST	Style d'API web standardise	Representational State Transfer -- stateless,
SPA	Application web sans rechargement	Single Page Application -- React router cote
ORM	Abstraction de la base de donnees	Object-Relational Mapper -- Sequelize mapp
Middleware	Filtre de requete HTTP	Fonction Express(req, res, next) -- intercept
Heuristique	Regle empirique approximative	Regle derivee de l'observation, pas d'entraîn
ELA	Detection de retouche image	Error Level Analysis -- artefacts JPEG revela
NER	Extraction d'entites nommees	Named Entity Recognition -- spaCy identifie
JSONB	JSON stocke efficacement	Binary JSON PostgreSQL -- indexable, requ
Rate limiting	Limite de debit reseau	Compteur par IP + fenetre glissante -- protec
UUID v4	Identifiant unique aleatoire	128 bits d'entropie -- non-devinable, pas de
Stateless	Sans etat cote serveur	Chaque requete contient toutes ses informa
Whitelist	Liste blanche autorisee	Inverse de blacklist -- seuls les elements list
Polymorphisme	Meme interface, types differents	Media appartient a Candidat OU Institut selo
Affichage optimiste	Reponse UI avant confirmation	UI mise a jour avant validation serveur pour
Hard cap	Plafond infranchissable	Score ne peut pas depasser X quelle que so
Graceful degradation	Degradation propre	Composant non critique en erreur -> valeur
Shadow run	Execution parallele pour comparaison	V6 et V7 calculent en parallele pendant la m

RESUME POUR SOUTENANCE -- A RETENIR ABSOLUMENT

Les 5 decisions architecturales cles a expliquer

1. Transactions ACID dans candidatureWorkflow.js : Toutes les etapes de soumission (profil, docs, OCR score, statut,

notifications) sont dans UNE transaction. Coherence ou rien.

2. JWT + verification DB a chaque requete : Le middleware ne fait pas confiance au seul JWT -- il verifie que l'utilisateur existe encore et n'est pas suspendu. Cout : 1 requete DB par requete. Benefice
3. OCR non-bloquant : La verification du diplome ne bloque pas la soumission. Si le microservice est down -> log warning -> soumission reussie. La plateforme de candidature ne doit pas dependre de la dis
4. Heuristiques explicables vs Deep Learning : Le scoring V7 produit 6 sous-scores avec interpretation metier claire. Un institut peut comprendre pourquoi un diplome est suspect. Pas de boite noire.
5. Whitelist anti-mass-assignment : CHAMPS_PROFIL_AUTORISES est une liste explicite des champs qu'un candidat peut modifier. Securite defensive : ignore silencieusement tout champ non liste.

Les 3 points forts a valoriser

1. Workflow complet de bout en bout : De l'invitation de l'institut jusqu'a la pre-inscription post-acceptation, en passant par la verification OCR -- le cycle de vie complet est implemente.
2. Verification documentaire V7 : Pipeline deterministe multi-score avec detection de templates officiels sans identite, tampering ELA/DCT, et scoring explicable.
3. i18n bilingue FR/EN : Toutes les chaines UI passent par i18next -- aucune chaine en dur dans les composants.

Les 3 limitations a admettre proactivement

1. Zero tests automatises -- premiere priorite d'evolution.
2. MantraNet desactive -- architecture prete, calibration sur corpus reel necessaire.
3. Refresh token non implemente -- JWT 7j acceptable MVP, colonne presente.

Document genere le 2026-05-18 pour la soutenance PFE EduBridge.

Contient : architecture ASCII, flux detailles, analyse code, 25+ questions jury, forces/faiblesses, vocabulaire.

10. CHATBOT EDUBRIDGE ASSISTANT -- ANALYSE COMPLETE

10.1 Vue d'ensemble

Le chatbot EduBridge est un assistant conversationnel a base de regles (rule-based), integre directement dans la plateforme. Il repond aux questions des candidats en francais ET en anglais, sans recours a un LLM (Large Language Model) ni a une API IA externe.

Composants impliquees :

```
CANDIDAT (navigateur)
  |
  v
ChatbotButton.tsx    <- Bouton FAB flottant (bottom-right)
  | ouvre
  v
ChatbotPanel.tsx     <- Interface chat (panel 380x560px)
  | POST fetch()
  v
/api/chatbot/message <- Route Express publique (sans authMiddleware)
  |
  v
chatbotController.js <- Thin controller (8 lignes)
  | appelle
  v
chatbotService.js    <- Moteur de matching d'intents (~1000 lignes)
  |
  v
{ text, suggestions, link } <- Reponse structuree JSON
```

Caracteristiques cles :

- Public : pas d'authentification requise, accessible aux visiteurs non connectes
- Cache pour les roles admin et institut (inutile pour eux)
- Bilingue FR/EN : chaque intent a deux versions de reponse
- Stateless : aucune session, aucun historique de conversation conserve en base
- Non-bloquant : fetch() direct sans passer par le client axios centralise

10.2 Architecture dossier chatbot

backend/routes/chatbotRoutes.js

Role : Route Express minimale -- 1 seule route publique.

```
router.post('/message', chatbotController.sendMessage);
// Montee dans index.js sous /api/chatbot
```

Pourquoi publique ?

Un visiteur non connecte doit pouvoir poser des questions sur les programmes et l'inscription avant de creer un compte. L'authMiddleware bloquerait cette UX.

Risque et mitigation :

La route publique est couverte par le rate limiter global (600 req/15min/IP). Aucune donnee sensible n'est retournee -- uniquement du texte statique.

backend/controllers/chatbotController.js

Role : Controller ultra-mince -- 8 lignes de logique.

Ce qu'il fait :

1. Extrait { message, lang } du body
2. Valide que le message n'est pas vide
3. Normalise la langue (lang === 'en' ? 'en' : 'fr', fallback FR)
4. Delege a chatbotService.processMessage()
5. Retourne { success: true, data: { response } }

Pattern applique : Thin Controller -- zero logique metier ici.

backend/services/chatbotService.js

Role : Coeur du chatbot -- base de connaissances + moteur de matching.

Structure du fichier :

```
chatbotService.js
+-- INTENTS[]          <- ~50 intents dans 11 categories
|  +-- { id, keywords[], response{}, response_en{} }
|  +-- ...
+-- FALLBACK_FR        <- Reponse par default si aucun match
+-- FALLBACK_EN        <- Idem en anglais
+-- processMessage()   <- Algorithme de scoring et selection
```

11 categories d'intents :

	Categorie	Exemples d'intents
1	Candidature & Dossier	apply_how, apply_status, apply_tracking, ap
2	Documents & Upload	docs_required, docs_diploma, docs_upload
3	Programmes & Recherche	programs_find, programs_engineering, prog
4	Instituts & Accreditations	institutes_list, accreditation_what, accredita
5	Compte & Authentification	auth_register, auth_login, auth_reset_passw
6	Guide plateforme	guide_how_platform, faq_account, faq_cand
7	Pre-inscription	post_accepted, post_interview, post_enrollm
8	Vie etudiante en Tunisie	life_cost, life_housing, life_transport, life_fo
9	Frais & Financement	fees_tuition, fees_payment, fees_scholarshi
10	Etudiants Internationaux	intl_visa, intl_residence
11	Post-Admission	post_accepted, post_rejected

Structure d'un intent :

```
{
  id: 'apply_how',
  keywords: ['comment', 'candidater', 'postuler', 'soumettre', 'apply', ...],
  response: {
    text: "Pour candidater sur EduBridge...",
    suggestions: ["Documents requis", "Trouver un programme"],
    link: { label: "Parcourir les programmes", path: "/search" }
  },
  response_en: {
    text: "To apply on EduBridge...",
```

```

    suggestions: ["Required documents", "Find a program"],
    link: { label: "Browse programs", path: "/search" }
  }
}

```

10.3 Algorithme de matching -- Analyse detaillee

```

function processMessage(rawMessage, lang = 'fr') {

  // -- 1. Nettoyage et tokenisation -----
  const cleaned = rawMessage
    .toLowerCase()          // "Comment CANDIDATER ?" -> "comment candidater ?"
    .trim()
    .replace(/[.,!?:'"()\[\]]/g, ' ') // retire la ponctuation
    .replace(/\s+/g, ' '); // normalise les espaces

  const words = cleaned.split(' ').filter(w => w.length > 2);
  // Filtre les mots trop courts ("le", "la", "de") -> moins de bruit

  // -- 2. Scoring de chaque intent -----
  let bestIntent = null;
  let bestScore = 0;

  for (const intent of INTENTS) {
    let score = 0;

    for (const word of words) {
      for (const keyword of intent.keywords) {

        if (word === keyword) {
          score += 3; // Match EXACT -- signal fort
          break;     // Un seul bonus par mot
        }
        if (word.includes(keyword) || keyword.includes(word)) {
          score += 1; // Match PARTIEL -- signal faible
          // Ex: "candidatures" includes "candidature" -> +1
          break;
        }
      }
    }

    if (score > bestScore) {
      bestScore = score;
      bestIntent = intent;
    }
  }

  // -- 3. Selection de la reponse -----
  if (bestScore === 0 || !bestIntent) {
    return lang === 'en' ? FALLBACK_EN : FALLBACK_FR;
    // Aucun mot reconnu -> reponse generique avec suggestions
  }

  if (lang === 'en' && bestIntent.response_en) {
    return bestIntent.response_en; // Version anglaise si demandee
  }
  return bestIntent.response;     // Version francaise par default
}

```

Exemple concret de scoring :

Message : "Comment soumettre mon dossier de candidature ?"

Tokens : ["comment", "soumettre", "mon", "dossier", "candidature"]

Tokens filtres > 2 chars : ["comment", "soumettre", "dossier", "candidature"]

```
Intent apply_how keywords: ['comment', 'candidater', 'soumettre', ...]
-> "comment" === "comment" +3
-> "soumettre" === "soumettre" +3
-> "dossier" inclus dans kw? +1 (partiel avec "dossier")
-> SCORE = 7
```

```
Intent docs_required keywords: ['documents', 'dossier', ...]
-> "dossier" === "dossier" +3
-> SCORE = 3
```

Meilleur : apply_how (score=7) -> reponse "Pour candidater sur EduBridge..."

Limites du matching par mots-cles :

- Pas de comprehension semantique : "quelle est la procedure pour envoyer mon dossier ?" peut rater si "procedure" et "envoyer" ne sont pas dans les keywords
- Pas de gestion du contexte conversationnel : chaque message est traite independamment
- Pas de correction orthographique : "candidaure" ne matche pas "candidature"

10.4 Interface utilisateur -- ChatbotPanel.tsx et ChatbotButton.tsx

Structure du panel

```
+-----+
| [Bot] EduBridge Assistant [X] | <- Header bleu #2563EB
| ? En ligne                    |
+-----+
|                               |
| [Assistant] Bonjour ! Je suis... | <- Bulle grise (gauche)
| [Candidater?] [Documents?] [Comparer?]| <- Suggestions chips
|                               |
| [Comment candidater?] [User] | <- Bulle bleue (droite)
|                               |
|                               | 10:32 |
|                               |
| [Assistant] Pour candidater... |
| [Documents requis] [Suivi] [Programme] |
| [-> Parcourir les programmes] | <- Link button
|                               |
| [***]                          | <- Typing indicator (anime)
+-----+
| [Candidater?] [Docs?] [Comparer?] ... | <- Quick actions (1er msg)
+-----+
| [ Posez votre question... ] [->] | <- Input + Send button
+-----+
```

Fonctionnalites UX remarquables

1. Message de bienvenue (useEffect ligne 64-69) :

```
useEffect(() => {
  if (isOpen && !hasWelcomed) {
    setMessages([buildWelcome()]);
    setHasWelcomed(true);
  }
}, [isOpen, hasWelcomed]);
```

-> S'affiche UNE seule fois par session. Si on ferme et rouvre le panel, le message de bienvenue ne reapparaît pas.

2. Reset bilingue (useEffect ligne 71-75) :


```
useEffect(() => {
  setHasWelcomed(false);
  setMessages([]);
}, [i18n.language]);
```

-> Quand l'utilisateur change de langue (FR->EN), la conversation est remise a zero et le message de bienvenue reapparaît dans la nouvelle langue.

3. Auto-scroll (useEffect ligne 78-80) :

```
useEffect(() => {
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
}, [messages, isLoading]);
```

-> Scroll automatique vers le bas a chaque nouveau message ET pendant le chargement (typing indicator).

4. Placeholder rotatif (useEffect ligne 82-88) :

Toutes les 3 secondes, le placeholder de l'input change parmi 4 suggestions de questions. Nettoyage du setInterval via le return de useEffect (pas de fuite memoire).

5. Typing indicator :

```
{isLoading && (
  <div>
    <span animate-bounce style={{ animationDelay: '0ms' }} />
    <span animate-bounce style={{ animationDelay: '150ms' }} />
    <span animate-bounce style={{ animationDelay: '300ms' }} />
  </div>
)}
```

3 points qui rebondissent en decalé de 150ms -- simulation de frappe.

6. Quick Actions :

Visibles UNIQUEMENT quand messages.length <= 1 (seulement le message de bienvenue). Disparaissent dès que le candidat pose sa première question pour libérer l'espace.

7. Suggestions chips :

Chaque réponse du chatbot peut inclure des suggestions[] cliquables. Un clic sur une suggestion appelle directement sendMessage(suggestion) -- pas de re-saisie.

8. Link button :

Chaque réponse peut inclure un link: { label, path }. Le clic ferme le panel ET navigue vers la route React correspondante.

ChatbotButton.tsx -- Bouton FAB

```
// Visible uniquement pour les candidats (et visiteurs)
if (user?.role === 'admin' || user?.role === 'institut') return null;

// Badge unread count (style WhatsApp)
{unreadCount > 0 && (
  <span className="absolute -top-1 -right-1 ... bg-red-500">
    {unreadCount > 9 ? '9+' : unreadCount}
  </span>
)}
```

Compteur de non-lus :

handleNewAssistantMessage() incremente unreadCount uniquement si le panel est FERME. Quand on ouvre le panel, unreadCount est remis a 0.

10.5 Flux complet message -> reponse

UTILISATEUR tape "comment candidater" + appuie Entree

```
|
v
```

```

handleKeyDown -> sendMessage(input)
  |
  v
Etat : setIsLoading(true)
  Ajoute bulle user a messages[]
  Vide l'input
  |
  v
fetch(`${API_URL}/chatbot/message`, {
  method: 'POST',
  body: JSON.stringify({ message: "comment candidater", lang: "fr" })
})
  |
  v HTTP POST
chatbotController.sendMessage
  -> chatbotService.processMessage("comment candidater", "fr")
  -> tokens: ["comment", "candidater"]
  -> best: apply_how (score=6)
  -> return response.text + suggestions + link
  |
  v JSON
{ success: true, data: { response: { text: "...", suggestions: [...], link: {...} } } }
  |
  v
addAssistantMessage({ text, suggestions, link })
setIsLoading(false)
  |
  v
UI : bulle assistant apparait + chips suggestions + bouton lien
  Auto-scroll vers le bas

```

10.6 Analyse code -- chatbotService.processMessage()

Nettoyage du message (lignes 955-960)

```

const cleaned = rawMessage
  .toLowerCase()
  .trim()
  .replace(/[.,!?:;"'()\[\]]/g, ' ') // punctuation -> espace
  .replace(/\s+/g, ' ');           // espaces multiples -> un seul

```

Pourquoi supprimer la ponctuation ?

"Comment candidater ?" -> sans nettoyage, "?" resterait colle a "candidater" et "candidater?" ne matcherait pas le keyword "candidater".

Filtre de longueur (ligne 961)

```
const words = cleaned.split(' ').filter(w => w.length > 2);
```

Filtre les mots de 1-2 caracteres : "je", "la", "le", "de", "un", "ma", "a"... Ces mots sont du bruit sans valeur semantique pour le matching d'intents.

Scoring dual (lignes 968-978)

```

if (word === keyword) {
  score += 3; break;           // Match exact : tres fort signal
}
if (word.includes(keyword) || keyword.includes(word)) {
  score += 1; break;           // Match partiel : signal faible
}

```

```
}

```

Asymetrie intentionnelle :

- `word.includes(keyword)` : "candidatures" contient "candidature" -> +1
- `keyword.includes(word)` : "cand" est contenu dans "candidater" -> +1

Le break apres chaque match evite de compter plusieurs fois le meme mot contre plusieurs keywords du meme intent.

Reponse structuree

```
{
  text: "Pour candidater...",           // Texte principal (multiline)
  suggestions: ["Docs ?", "Statut ?"], // 2-3 boutons suggeres
  link: { label: "...", path: "/"... } // Navigation contextuelle
}
```

La reponse text utilise `\n` pour les sauts de ligne -- le composant React les affiche avec `whitespace-pre-wrap`.

10.7 Questions probables du jury sur le chatbot

Q : Pourquoi un chatbot rule-based et non un LLM comme ChatGPT ?

Reponse courte : Controle total, determinisme, cout zero, zero dependance externe.

Reponse longue soutenane :

"Plusieurs raisons justifient ce choix. Premierement, un LLM necessite une API payante (OpenAI, Anthropic, Mistral) avec des couts variables difficiles a budgetiser pour un MVP. Deuxiemement, les reponses d'un LLM sont non-deterministes -- la meme question peut donner des reponses differentes, certaines potentiellement incorrectes sur les programmes ou les instituts specifiques a la Tunisie. Troisiemement, la confidentialite : les messages des candidats (nom, situation academique) transiteraient par un serveur externe. Notre approche rule-based garantit que les reponses sont 100% controlees, auditables et exactes. Le domaine est suffisamment borne (candidature, programmes, vie etudiante) pour etre couvert efficacement par ~50 intents."

Q : Quelles sont les limites de votre algorithme de matching ?

Reponse honnete :

"Notre algorithme souffre de trois limitations principales. Premierement, l'absence de correction orthographique : 'candidaure' ne matche pas 'candidature'. Deuxiemement, l'absence de contexte conversationnel : chaque message est analyse independamment, donc une question de suivi comme 'et pour les frais ?' sans contexte retourne une reponse generique. Troisiemement, les synonymes non listes : 'postuler a une ecole' peut rater si 'ecole' n'est pas dans les keywords. Une amelioration naturelle serait d'utiliser un moteur de NLP leger comme spaCy pour la lemmatisation et l'expansion de synonymes, sans aller jusqu'a un LLM."

Q : Pourquoi la route chatbot est-elle publique ?

Reponse :

"Le chatbot sert principalement a guider des visiteurs non inscrits : 'Quels programmes y a-t-il ?', 'Comment creer un compte ?', 'Quels sont les frais ?'. Forcer l'authentification avant de pouvoir poser ces questions creerait une friction inutile et decouragerait les inscriptions. La securite est assuree par le rate limiter global (600 req/15min/IP) -- il est impossible de floodier le service depuis une seule IP. Le service ne retourne aucune donnee sensible, uniquement du texte statique pre-ecrit."

Q : Pourquoi le chatbot n'utilise pas le client axios centralise de api.ts ?

Reponse :

"Le chatbot est une fonctionnalité accessible aux visiteurs non authentifiés. Le client axios centralise dans api.ts a un intercepteur de reponse qui redirige vers /login sur une erreur 401. Si un visiteur non connecte utilisait ce client, une erreur reseau temporaire pourrait declencher cette redirection de facon inattendue. En utilisant fetch() natif directement, on evite ce comportement. C'est un choix delibere pour isoler la logique du chatbot de l'infrastructure d'authentification."

Q : Pourquoi le chatbot est-il masque pour les roles admin et institut ?

Reponse :

"Le chatbot repond aux questions de candidature, programmes et vie etudiante -- des informations utiles uniquement pour les candidats. Un admin a le dashboard de gestion, un institut a son dashboard de candidatures. Afficher le chatbot pour ces roles creerait de la confusion dans l'interface et du bruit inutile. La logique est dans ChatbotButton.tsx : if (user?.role === 'admin' || user?.role === 'institut') return null; -- le composant n'est tout simplement pas monte."

Q : Comment etendriez-vous le chatbot si vous aviez plus de temps ?

Reponse :

"Trois axes d'evolution. Premierement, le contexte conversationnel : conserver l'historique des 3 derniers echanges et l'envoyer avec chaque nouvelle requete pour permettre les questions de suivi ('et pour les frais ?' apres 'quels programmes a Sousse ?'). Deuxiemement, la lemmatisation : utiliser compromise.js ou un dictionnaire de synonymes pour 'postuler' = 'candidater' = 'soumettre'. Troisiemement, les analytics : logger les messages sans reponse (score=0) pour identifier les lacunes et enrichir les intents. A terme, si le trafic justifie l'investissement, un LLM fine-tune sur les donnees EduBridge remplacerait le matching par mots-cles."

10.8 Forces et faiblesses du chatbot

Forces

Force	Explication
Gratuit et stateless	Aucune API externe, aucune base de donnees, aucun cout
Deterministe	Meme question = meme reponse toujours
Bilingue natif	Chaque intent a FR et EN, commutation sans re-apprentis
Reponse structuree	text + suggestions + link = UX guidee, pas juste du tex
Non-bloquant	Si le backend plante, l'erreur est capturee et une sugg
Scope borne	~50 intents couvrent 95% des questions d'un candidat tu
Reset bilingue	Conversation remise a zero proprement au changement de

Faiblesses honnetes

Faiblesse	Degre	Mitigation
Pas de contexte	Eleve	Chaque message est independant
Pas de correction ortho	Moyen	Suggestions chips guident vers les bonnes
Synonymes non couverts	Moyen	Keywords etendus manuellement par cate

Maintenance manuelle	Moyen	Ajout d'un intent = modification du code
Pas d'analytics	Faible	On ne sait pas quelles questions sont non r

Comment defendre intelligemment

"Votre chatbot est trop simple, pourquoi pas GPT ?"

"Pour un MVP sur un domaine borne, un chatbot rule-based est plus fiable et plus controlable qu'un LLM. Nos reponses sont exactes sur les programmes tunisiens parce qu'elles sont ecrites manuellement. Un LLM pourrait halluciner des informations sur les frais, les accreditations ou les dates limites -- ce qui serait prejudiciable pour des candidats qui prennent des decisions d'orientation serieuses."

10.9 Diagramme d'etat -- Conversation chatbot

```

VISITEUR ARRIVE
|
v
[ChatbotButton visible]
(masque si admin/institut)
| clic
v
Panel ouvert -> Message bienvenue
+ 6 Quick Actions
|
|--- Clic quick action -----+
|                               |
| Saisie texte + Entree        |
|                               |
v                               v
isLoading = true               sendMessage(quickAction)
bulle user ajoutee             |
input vide                     |
|                               |
v                               |
POST /api/chatbot/message <-----+
|
v
processMessage() -> scoring -> bestIntent
|
+-- score > 0 -> response du bestIntent
+-- score = 0 -> FALLBACK (reponse generique)
|
v
addAssistantMessage({ text, suggestions, link })
isLoading = false
Auto-scroll
|
+-- Clic suggestion -> sendMessage(suggestion) [boucle]
+-- Clic link button -> navigate(path) + ferme panel
+-- Saisie nouvelle question [boucle]

```

10.10 Recapitulatif chatbot pour la soutenance

A retenir absolument

1. Rule-based, pas LLM : scoring par mots-cles (+3 exact, +1 partiel), ~50 intents, 11 categories, ~1000 lignes de base de

connaissances.

2. Route publique intentionnelle : visiteur non connecte -> peut poser des questions -> reduit la friction a l'inscription.
3. Bilingue natif : pas de traduction automatique, deux versions manuelles par intent pour garantir la qualite.
4. Reponse structuree : { text, suggestions[], link } -> UX guidee avec navigation contextuelle integree.
5. Stateless : aucune conversation en base, aucun historique -- simplifie le deploiement et la scalabilite.
6. Masque admin/institut : ChatbotButton ne se monte pas pour ces roles -- UX propre.

Chiffres a citer en soutenance

- ~50 intents dans 11 categories
- 2 langues : FR et EN, 2 versions manuelles par intent
- 1 seule route : POST /api/chatbot/message
- 0 base de donnees : stateless
- 0 API externe : self-hosted, cout variable = 0
- Score max d'un intent = nb_mots_communs x 3 (matches exacts)

Section chatbot ajoutée le 2026-05-19.